

Concurrent Task Dispatcher

Chris Sanchez - 3334-01-Spring2026

Threads, Components, and Shared Data

There are five components in the system that work concurrently. Using a fixed random seed, the generator thread creates tasks, assigns each one a kind and a current arrival timestamp, and then sends them one at a time over an MPSC channel with a fixed 20 ms sleep in between. When each task has been sent, the generator drops the sender and closes the channel. Working on the main experiment thread, the manager is the sole decision-maker. Keeping an internal Vec queue, it will get tasks coming from the generator channel and decide if each task may be dispatched based on two factors. The cost of the task plus the current global CPU percentage and whether a free worker is not being used at 100 percent. Tasks that cannot be dispatched stay in the queue until a worker finishes and signals that its available.

Threads share three values. A single `Arc<Mutex<SharedState>>` handles the `active_workers`, `cpu_pct`, and `queue_len` stored in a `SharedState`. It is read and written by the manager before to every dispatch decision, decremented by workers on its completion, and read by the monitor every 10 milliseconds. The submitted counter, that the main thread uses to figure out how many records to gather, is an `Arc<Mutex<u64>>` that is increased by the manager each time a job is sent out. To allow all eight workers to share access to the same incoming task stream, the worker channel receiver is wrapped in `Arc<Mutex<Receiver>>`.

Additionally, the threads share three pieces of state. To enable all eight workers to pull from the same channel, the pool receiver is wrapped into `Arc<Mutex<Receiver<Message>>`. Workers are operating concurrently because mutex is only kept during `recv()`, not during execution. The dispatcher and main thread share an `Arc<AtomicU64>` called `submitted_count`. Since only one increment and one load are required across the join boundary, an atomic is adequate. An `Arc<AtomicUsize>` called `active_workers` was introduced as a third variable. It is increased when a worker takes on a task and decreased when it is completed. This will eliminate the need for extra locks and enable the monitor thread to view the pool pressure in real time.

Channels and why they were chosen

Four channels are used. An unbounded mpsc channel carries `Task` structs from the generator to the manager. It is unbounded because tasks arrive at a controlled 20ms rate and will not grow without bound. A bounded sync_channel with capacity 32 carries task tuples from the manager to the workers. The bound limits how far ahead the manager can dispatch and prevents

memory from growing if workers fall behind. An unbounded mpvc channel carries CompletionRecord from all eight workers back to the main thread. It closes automatically when every worker's done_tx clone is dropped, which signals the main thread that the collection is complete. The fourth unbound channel then carries empty release signals from workers back to the manager. Each time a worker finishes the given task it will then send on this channel, which will wake up the manager and if it is blocked waiting for a slot to open. This avoids the manager from spinning all the workers when they are occupied.

Scheduling Policy and Trade-offs

Two policies are implemented and compared with the same 1000 task workload. Firstly, the FIFO which uses a manager that holds tasks in a Vec and always dispatches from index zero, preserving arrival order. Before being dispatched, it will check the active_workers is below eight and that cpu_pct plus the task's cost does not exceed 100 percent. If any of these checks fail, the manager will block the release channel until a worker finishes its job. FIFO guarantees that no task is skipped in favor of another and distributes the work evenly.

The second policy is an optimized approach. Instead of single queue the manager maintains two separate Vecs, one for CPU Tasks and one for IO Tasks. With every dispatch cycle it will check available CPU headroom and prefers CPU tasks when about 35 percent is free, falling back to IO tasks when headroom is more constricted. This will allow the manager to pack more concurrent workers into the available CPU and when CPU tasks are running and consuming 35 percent each, the remaining headroom can still take in IO Tasks at a 10 percent each. Therefore, keeping workers busy at the same time, both policies enforce the same hardcap of 100 percent total CPU and the same limit of eight workers.

The weakness that follows FIFO is head of line blocking. If several CPU tasks arrive at the same time, they will consume 35 percent each, and at three concurrent tasks that cap would hit 105 percent. Thus, the manager can only run two CPU tasks at once without breaking the bank, leaving workers idle even when IO tasks are queued and could fit. The optimized policy on the other hand cover this problem by explicitly filling remaining headroom with IO Tasks. The trade off is the added complexity in the dispatch loop and slightly less strict order fairness, since IO Tasks may be dispatched ahead of CPU tasks that arrived a bit earlier.

Bugs Encountered and Fixed

So when i first started developing the simulation i had trouble with simulate_cpu_work for a bit. This function was originally used in a wrapping arithmetic operation inside a busy loop whose results were never used, causing CPU tasks to complete in near-zero time instead of the intended 200ms. The original version would assign the counter result to an underscore binding with let _ = counter, which in Rust is a hint to discard the value and gives the compiler permission to pretty much eliminate the computation entirely.

The fix i had in mind and with the help of Claude.ai was to keep the wrapping_add call but change the final line to let `_ = counter`, which reads the variable in a side effectful binding that prevents dead-code eliminating itself without requiring any external crate or unsafe hints. Testing confirmed that CPU tasks consistently ran for the 200ms after the change. This was an important metric to work out because of the accuracy required for CPU tasks because if the tasks are completed instantly, the wait and turnaround times would be meaningless and the CPU percentage tracking in SharedState would never be stressed as originally intended.

Remaining Starvation and Unfairness

Under the FIFO Policy, CPU Heavy bursts can cause IO tasks to arrive mid-burst to wait far longer than their 200ms time. With three CPU tasks running at the same time the CPU budget would hit 105 percent when rounded, so the manager cannot dispatch anything until a CPU task finishes. IO Tasks that arrived during the time slot are not permanently starved but their wait till will drastically increase relative to their cost. The optimized path on the other hand reduces this but does not eliminate it entirely, since CPU tasks still take priority when headroom is available and may even delay IO tasks that arrive early.

A second source of unfairness that was noted was the shared worker channel receiver. All eight workers lock the same Mutex to receive a new task, and the OS decides which worker wins the lock each time. With over 1000 tasks this tends to balance itself out, but in shorter runs some workers may process a greater amount of tasks than others due to scheduling differences. The current design has no fallback to detect or correct this imbalance. Per worker queues with a work stealing mechanism would address both of the issues presented but would greatly increase the complexity of the manager and worker code.